

**NED UNIVERSITY OF ENGINEERING AND
TECHNOLOGY**

ASSIGNMENT SBSDI:

Certificate Issuance Smart Contract

Submitted to:

Dr. Kashif Mehboob

Submitted by:

Usman Rasheed Siddiqui (SBSDI-NED-179)

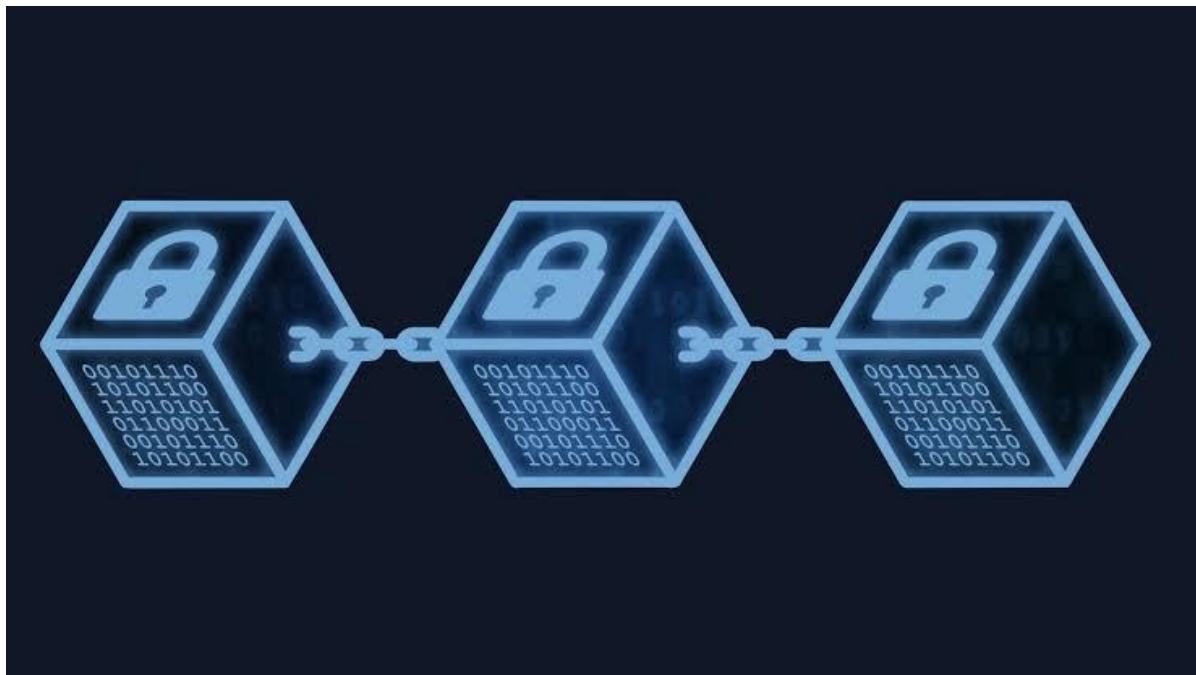


Table of Contents

1. Project Overview	3
2. System Workflow	3
3. Smart Contracts and Functions	3
3.1 Citizen.sol	3
3.2 Skills.sol	3
3.3 Validation.sol	4
3.4 Certificate.sol	4
4. Certificate Types	4
5. Deployment – Step by Step	4
6. Deployment Information	5
7. GitHub Repository	5

1. Project Overview

The Skills-Based Certificate Issuing System is an Ethereum-based smart contract system for issuing skill certificates to eligible citizens. It stores citizen information and skill scores, validates certificate eligibility, and records certificates on the blockchain.

The system contains four connected Solidity contracts: Citizen, Skills, Validation, and Certificate. The Government manages the system, while an Authorized Third Party can add citizen/skill data and issue certificates.

2. System Workflow

1. Citizen data is added or updated through Citizen.sol.
2. Skill scores are stored through Skills.sol.
3. Validation.sol checks age, Intermediate qualification, and required skill scores.
4. Certificate.sol issues a certificate only when the required validation condition is satisfied.
5. The Government can revoke an active certificate and reactivate a revoked certificate when no other active certificate of that type exists.

3. Smart Contracts and Functions

3.1 Citizen.sol

Purpose: Stores citizen identity and eligibility information.

Function	Access	Description
<code>addCitizen(name, cnic, age, intermediatePassed)</code>	Authorized Third Party	Adds a new citizen record.
<code>updateCitizen(name, cnic, age, intermediatePassed)</code>	Authorized Third Party	Updates an existing citizen record.
<code>getCitizen(cnic)</code>	Public / View	Returns the citizen's name, CNIC, age, Intermediate status, and existence.
<code>citizenExists(cnic)</code>	Public / View	Checks whether a citizen record exists.
<code>setAuthorizedThirdParty(newThirdParty)</code>	Government	Changes the authorized third-party address.

3.2 Skills.sol

Purpose: Stores skill scores from 0 to 100 for Graphic Designing and Web Development.

Function	Access	Description
<code>setGraphicDesigningSkills(cnic, adobeCreativeTool, figma, premiere, photoshop)</code>	Authorized Third Party	Stores or updates four Graphic Designing skill scores.
<code>setWebDevelopmentSkills(cnic, react, typescript, django, api)</code>	Authorized Third Party	Stores or updates four Web Development skill scores.
<code>getGraphicDesigningSkills(cnic)</code>	Public / View	Returns the citizen's Graphic Designing skill scores.
<code>getWebDevelopmentSkills(cnic)</code>	Public / View	Returns the citizen's Web Development skill scores.
<code>setAuthorizedThirdParty(newThirdParty)</code>	Government	Changes the authorized third-party address.

3.3 Validation.sol

Purpose: Determines whether a citizen satisfies the certificate eligibility conditions.

Function	Access	Description
<code>citizenIsValid(cnic)</code>	Public / View	Checks that the citizen exists, is at least 18 years old, and has passed Intermediate.
<code>graphicDesigningCertificateCondition(cnic)</code>	Public / View	Checks citizen eligibility plus Adobe Creative Tool and Photoshop scores of at least 60.
<code>webDevelopmentCertificateCondition(cnic)</code>	Public / View	Checks citizen eligibility plus React and Django scores of at least 60.

Important constants: `PASSING_SCORE = 60` and `MINIMUM_AGE = 18`.

3.4 Certificate.sol

Purpose: Issues, verifies, revokes, and reactivates skill certificates.

Function	Access	Description
<code>setAuthorizedThirdParty(address)</code>	Government	Sets the address allowed to issue certificates.
<code>issueCertificate(cnic, certificateType)</code>	Authorized Third Party	Validates eligibility and creates a new certificate with an automatically generated ID.
<code>revokeCertificate(certificateId)</code>	Government	Deactivates an active certificate and removes it from the active-certificate mapping.
<code>reactivateCertificate(certificateId)</code>	Government	Reactivates a revoked certificate if no other active certificate of the same type exists.
<code>verifyCertificate(certificateId)</code>	Public / View	Returns the citizen name, CNIC, certificate type, issue date, and active status.
<code>certificates(certificateId)</code>	Public Getter	Returns the stored certificate record for the given ID.
<code>activeCertificate(cnic, certificateType)</code>	Public Getter	Returns the currently active certificate ID for a citizen and certificate type; 0 means none.

4. Certificate Types

Value	Certificate Type	Required Skills
0	Graphic Designing	Adobe Creative Tool >= 60 and Photoshop >= 60
1	Web Development	React >= 60 and Django >= 60

Example: `issueCertificate(123456789, 0)` issues a Graphic Designing certificate, while `issueCertificate(123456789, 1)` issues a Web Development certificate. The certificate ID is generated automatically.

5. Deployment – Step by Step

Step 1 – Prepare the contracts: Open Remix IDE and create the four Solidity files: `Citizen.sol`, `Skills.sol`, `Validation.sol`, and `Certificate.sol`. Use the Solidity compiler version required by the source code (the deployed version used `solc 0.8.34`).

Step 2 – Compile `Citizen.sol`: Compile `Citizen.sol` in Remix. Make sure there are no compilation errors. Copy the deployed contract address after deployment.

Step 3 – Deploy `Citizen.sol`: Deploy `Citizen.sol`. The Government address is set from the deploying account according to the contract constructor/implementation. Save the deployed address.

Step 4 – Deploy `Skills.sol`: Deploy `Skills.sol` and save its contract address. Set the authorized third party using `setAuthorizedThirdParty(newThirdParty)` from the Government account.

Step 5 – Configure Citizen.sol: From the Government account, call `setAuthorizedThirdParty(newThirdParty)` in `Citizen.sol` so the authorized account can add and update citizens.

Step 6 – Deploy Validation.sol: Deploy `Validation.sol` using the `Citizen` contract address and `Skills` contract address required by its constructor. Save the `Validation` contract address.

Step 7 – Deploy Certificate.sol: Deploy `Certificate.sol` using the `Validation` contract address and `Citizen` contract address as constructor parameters. Save the `Certificate` contract address.

Step 8 – Configure Certificate.sol: From the Government account, call `setAuthorizedThirdParty(newThirdParty)` so the authorized account can issue certificates.

Step 9 – Add a citizen: Switch to the Authorized Third Party account and call `addCitizen(name, cnic, age, intermediatePassed)` in `Citizen.sol`.

Step 10 – Add skills: Use the Authorized Third Party account to store the required Graphic Designing or Web Development skill scores in `Skills.sol`.

Step 11 – Issue certificate: Call `issueCertificate(cnic, certificateType)` in `Certificate.sol`. Use 0 for Graphic Designing and 1 for Web Development. The `Validation` contract checks eligibility before the certificate is created.

Step 12 – Verify certificate: Call `verifyCertificate(certificateId)` to view the certificate's citizen name, CNIC, type, issue date, and active status.

Step 13 – Revoke / reactivate: The Government can call `revokeCertificate(certificateId)`. A revoked certificate can be reactivated using `reactivateCertificate(certificateId)` if the contract's active-certificate condition allows it.

6. Deployment Information

The deployed `Certificate` contract was verified on Sourcify on the Sepolia test network.

Item	Value
Network	Sepolia
Chain ID	11155111
Certificate Contract	0x1e5378bF63F636B3Afe4FA26BdC44a199FAB34aD
Compilation Target	contracts/Certificate.sol:Certificate
Compiler	solc 0.8.34

Note: Replace the deployment information above if the contracts are redeployed and new addresses are generated.

7. GitHub Repository

GitHub Repository: <https://github.com/Usman-Rasheed-Siddiqui/Certificate-Issuance-Smart-Contract>